

Day 1 Lab: Real OAuth Flow With Go

Goal

Participants see a real runnable OAuth-like flow in Go and understand every important request.

This lab is not asking participants to create code from scratch. The instructor runs the demo, opens the browser, and explains how the flow works.

1. Run the App

From this folder:

```
go run .
```

Open:

<http://localhost:8080>

2. What Is Running

This single Go app simulates three systems:

System	Local Route Prefix	Meaning
Client App	<code>/client/*</code>	The app asking for API access
Authorization Server	<code>/auth/*</code>	Issues authorization codes and tokens
Resource Server API	<code>/api/*</code>	API that receives access tokens

In real life, these are often separate systems. In this demo, they are one program so the flow is easy to teach.

3. Authorization Code + PKCE Flow

Click:

Start Auth Code + PKCE flow

The client creates:

- `state`
- `code_verifier`
- `code_challenge`

Then it redirects the browser to:

`/auth/authorize`

Important authorization request parameters:

Parameter	Meaning
<code>response_type=code</code>	Ask for an authorization code
<code>client_id=repoboard-web</code>	Identify the client app
<code>redirect_uri</code>	Where the browser returns after approval
<code>scope=profile.read</code>	Permission requested
<code>state</code>	Client value checked on callback
<code>code_challenge</code>	PKCE challenge sent before the code exists
<code>code_challenge_method=S256</code>	Challenge uses SHA-256

4. User Approval

Click:

Approve access

The authorization server creates an authorization code and redirects back to:

`/client/callback?code=...&state=...`

Instructor explanation:

- `code` is not the access token.
- `state` must match what the client originally created.
- The browser sees the code, but not the access token.

5. Token Exchange

The client callback exchanges the code by sending a back-channel request to:

POST `/auth/token`

Important token request fields:

Field	Meaning
<code>grant_type=authorization_code</code>	Use Authorization Code Flow
<code>code</code>	Short-lived code from callback
<code>redirect_uri</code>	Must match the original request
<code>client_id</code>	Client exchanging the code
<code>code_verifier</code>	Original PKCE secret

Why send `redirect_uri` to `/token`?

The token endpoint does not redirect the browser. It uses `redirect_uri` to verify that this token exchange matches the original authorization request.

6. PKCE Explanation

PKCE uses this idea:

```
code_challenge = BASE64URL(SHA256(code_verifier))
```

The authorization server stores the challenge.

Later, the token endpoint receives the verifier, hashes it, and compares it with the original challenge.

If an attacker steals only the authorization code, they still do not know the `code_verifier`.

7. Call the Resource API

Click:

Call Resource API with current access token

The client calls:

```
GET /api/profile
```

```
Authorization: Bearer access_token_here
```

The Resource Server checks:

- Is the token known?
- Is it expired?
- Does it have `profile.read` scope?

8. Refresh Token Flow

Click:

Use refresh token

The client calls:

```
POST /auth/token
```

```
grant_type=refresh_token
```

```
refresh_token=...
```

```
client_id=repoboard-web
```

This demo rotates the refresh token:

- old refresh token is deleted,

- new access token is issued,
- new refresh token is issued.

9. Client Credentials Flow

Click:

Run Client Credentials demo

The backend service calls:

POST /auth/token

Authorization: Basic base64(order-service:order-service-secret)

grant_type=client_credentials

scope=fraud.check

Important:

- No user.
- No browser redirect.
- Token represents the service/client, not a human user.

10. Device Authorization Flow

Click:

Start Device Authorization demo

The client receives:

- device_code
- user_code
- verification_uri

The user opens the verification page and approves the device.

Then the client polls:

POST /auth/token

grant_type=urn:ietf:params:oauth:grant-type:device_code

device_code=...

client_id=deploy-cli

Before approval, the token endpoint returns:

```
{
  "error": "authorization_pending"
}
```

After approval, the token endpoint returns an access token.

11. Teaching Questions

Ask participants:

1. Which route is the authorization endpoint?
2. Which route is the token endpoint?
3. Which request goes through the browser?
4. Which request is back-channel?
5. Why is `state` checked?
6. Why is `code_verifier` sent only to `/auth/token`?
7. Why is Client Credentials not user login?
8. Why does Device Authorization need polling?

12. Source Code Reading Guide

Start from these functions in `main.go`:

Function	What to Explain
<code>ClientStartAuthCode</code>	Creates <code>state</code> , <code>code_verifier</code> , <code>code_challenge</code>
<code>AuthAuthorize</code>	Shows consent page
<code>AuthApprove</code>	Creates authorization code
<code>ClientCallback</code>	Checks <code>state</code> , exchanges code
<code>tokenAuthorizationCode</code>	Verifies code, redirect URI, and PKCE
<code>ClientCallAPI</code>	Calls Resource Server with bearer token
<code>tokenRefresh</code>	Demonstrates refresh token rotation
<code>tokenClientCredentials</code>	Demonstrates service-to-service token
<code>AuthDeviceAuthorization</code>	Starts Device Authorization Flow
<code>tokenDeviceCode</code>	Polling and token issuance

13. Key Takeaway

OAuth is easier when participants can see the real handoff:

`browser redirect -> code -> token endpoint -> access token -> API`

The access token is used at the Resource Server. The authorization code is only used to obtain tokens.